
بسم الله الرحمن الرحيم

Sharif University of Technology

Department of Electrical Engineering

Bachelor's Thesis

BME

Title:
Smoke and Fire Detection Project Using CCTV Cameras

Prepared by:

Armin Takbiri

Supervisor:

Dr. Narges-olhoda Mohammadzadeh

March 2026

Contents

List of Figures

List of Tables

Abstract

Early fire detection in industrial environments is crucial for preventing irreversible financial losses and threats to human life. In this work, an intelligent model based on Convolutional Neural Networks (CNN) is designed for accurate, real-time detection of smoke and fire. To achieve an efficient trade-off between inference speed and detection accuracy, the advanced YOLOv11 architecture is used as the core of the network. Given the challenge of limited high-quality labeled data, a comprehensive, industrial-oriented dataset was developed. This dataset is hybrid: part of it consists of available real-world videos, while the remaining part is composed of synthetic images generated by Generative AI models. Using synthetic data increases scenario diversity and improves the model's generalization ability in recognizing different smoke patterns. Finally, the proposed model was evaluated on diverse test sets; the results indicate high accuracy and reliable performance under varying environmental conditions.

Keywords: Smoke and fire detection; deep learning; YOLOv11; generative models; convolutional neural networks; dataset

Acknowledgements

With sincere gratitude and appreciation, we would like to thank our respected supervisor, Dr. Mohammadzadeh, for her valuable guidance, compassionate follow-up, and scientific support throughout all stages of this project.

We also thank Ms. Fakhfour, a Ph.D. student in Dr. Mohammadzadeh's group, whose suggestions, careful feedback, and scientific collaboration greatly helped us advance this work.

Finally, we are grateful to everyone who supported and accompanied us in any way during this project.

Chapter 1

Introduction

1.1 Background

Fire safety in industrial and workshop environments—especially in large spaces such as sheds, warehouses, and factories—is one of the most important requirements for safe and sustainable operation. A fire incident in such environments can rapidly lead to extensive financial losses, production downtime, damage to equipment and infrastructure, and in some cases direct threats to human life. In addition, the presence of flammable materials, machinery, industrial electrical systems, and continuous production processes makes the risk of ignition and fire spread inherently high in many industries. Therefore, early detection, as one of the most effective risk-reduction strategies, plays a key role in crisis management and in increasing the chance of containing a fire in its initial stages. This stage is often referred to as the "golden response time," where an alarm issued a few seconds to a few minutes earlier can make the difference between rapid containment and an industrial disaster.

1.2 Challenges and limitations of conventional methods

In conventional approaches, fire detection systems mainly rely on smoke, flame, or temperature sensors. These sensors are often installed at high elevations, and fully covering a large space typically requires many sensor units deployed across different locations. This requirement increases the initial procurement cost and also imposes significant additional costs, including installation, wiring, calibration, periodic testing, maintenance, and operational downtime during servicing. In industrial environments with high ceilings, dust, process vapors, strong ventilation, and installation constraints,

achieving full sensor coverage becomes even more challenging and may also increase the rate of false alarms.

Beyond cost and maintenance, one of the most important limitations in many industrial environments is the inherent detection delay. When a fire starts near the ground, behind machinery, or in a corner of a shed, smoke must rise to upper areas before it is detected by ceiling-mounted sensors. This delay can be amplified in very large spaces or in the presence of strong airflow (fans and exhaust systems), resulting in the loss of golden response time—when rapid extinguishing and preventing fire spread are most likely to succeed. As a result, alongside physical sensors, there is a need for solutions that can detect early indicators faster and closer to the ignition location.

1.3 Computer vision and AI-based solutions

Given the above limitations, computer vision and artificial intelligence offer an alternative path and also a complementary approach to conventional systems. In many factories, workshops, and warehouses, CCTV cameras are already widely deployed for monitoring and security purposes, providing an existing visual infrastructure. Leveraging this infrastructure for smoke and fire detection enables continuous environmental monitoring and the identification of incidents at the earliest stages of smoke formation. In this approach, instead of relying on smoke reaching high-mounted sensors, the image itself is analyzed and visual cues of smoke and fire are extracted—cues such as changes in texture and transparency, motion patterns, illumination changes, and the emergence of characteristic flame colors. Therefore, alarms can be issued faster, enabling more effective suppression, safer evacuation, and better prevention of fire spread.

Another advantage of this approach is reduced development and maintenance cost compared to expanding physical sensor deployments. Since cameras are often already installed, the need for additional hardware is reduced and the focus shifts toward designing and deploying a reliable software model. However, there is a fundamental difference between smoke sensors and image-based systems: smoke sensors are not sensitive to the visual appearance of smoke and only detect particles or physical changes, whereas computer vision models must distinguish smoke from its visual properties (shape, transparency, texture, color, and motion) against the background and similar-looking phenomena. Phenomena such as steam, fog, dust, reflective lighting, or shadows may appear visually similar to smoke, and if the training data is not sufficiently diverse, the probability of errors increases. Therefore, the accuracy and generalization of such sys-

tems strongly depend on the quality of the training data, coverage of environmental scenarios, and the robustness of the model under different conditions.

1.4 Research objective and report structure

Accordingly, the goal of this work is to present a deep learning-based framework for early smoke and fire detection using CCTV images—one that can identify smoke at different scales and shapes, under varying lighting conditions, and in diverse industrial environments, while also supporting practical deployment and fast response. To this end, object detection models based on Convolutional Neural Networks (CNN) are used so that relevant visual features of smoke and fire are automatically extracted and near real-time decision-making becomes possible. Moreover, given the common limitation of labeled data in industrial domains, data engineering and designing a dataset aligned with real-world conditions is a key pillar of this work, ensuring that the final model performs robustly across diverse scenarios.

The remainder of this report first explains the foundations and architecture of the proposed model and the rationale behind choosing it. Next, the process of data collection, labeling, and strategies for increasing data diversity (including synthetic data where needed) are described. Finally, training and evaluation results across different scenarios are reported and analyzed, clarifying the path for future improvements and practical deployment requirements.

Chapter 2

Fundamentals of Convolutional Networks and Choosing a Detection Architecture

2.1 Overview

This chapter reviews the fundamentals of convolutional neural networks and their core components in order to motivate an appropriate architecture choice for smoke and fire detection. Since the present problem requires online/real-time detection on CCTV videos, the selected architecture must offer not only adequate accuracy but also sufficient speed and computational efficiency. Accordingly, we first explain the structure of CNNs and their main building blocks, then review classic multi-stage approaches such as R-CNN as an influential baseline, and finally describe the rationale for selecting one-stage architectures such as YOLO.

2.2 Convolutional neural networks and the role of kernels

In images, neighboring pixels exhibit strong spatial correlation, and many patterns (edges, textures, shapes) appear locally. Fully connected networks typically face two serious issues when applied to images: (1) an extremely large number of parameters and high computational cost, and (2) ignoring the spatial structure of the input. Convolutional networks address these issues using the ideas of weight sharing and local receptive fields: a small filter (kernel) slides over the image and produces feature maps.

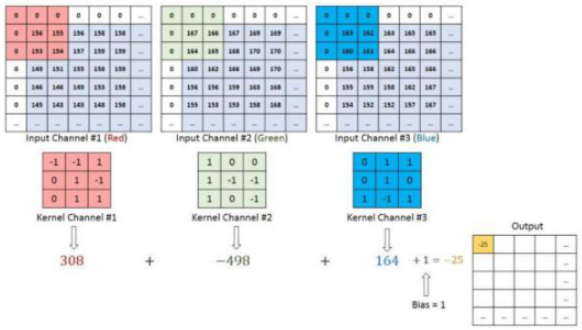


Figure 2.1: An example of the convolution operation: applying kernels to input channels and producing a feature map with an added bias term.

2.3 Main building blocks of a convolutional network

2.3.1 Convolution layer

Convolution can be viewed as a sliding inner product whose output (feature map) reflects the filter response to local patterns. By increasing the number of filters, the network can extract a wider variety of features; and by increasing depth, features evolve from simple patterns (edges) to higher-level patterns (more complex structures).

2.3.2 Nonlinear activation functions

Convolutional and fully connected layers are inherently linear mappings. If no nonlinear function is placed between layers, the entire network—regardless of depth—reduces to an equivalent linear mapping and loses the ability to model complex patterns. For instance, if

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = z^{(l)},$$

then composing multiple layers remains linear and can be collapsed into a single matrix. Therefore, adding an activation $\phi(\cdot)$ as

$$a^{(l)} = \phi\big(z^{(l)}\big)$$

is essential to increase model capacity and enable class separation in feature space.

Sigmoid and hyperbolic tangent

Two classic activations are:

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \textit{tanh}(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

While these functions map outputs to bounded ranges and are useful in some output layers (e.g., probabilities), their derivatives become small in saturated regions (large $|x|$), causing the Vanishing Gradient phenomenon and making deep network training more difficult.

ReLU and the "dead neuron" problem

The ReLU function is defined as

$$\textit{ReLU}(x) = \textit{max}(0, x).$$

Due to its simplicity and the lack of saturation in the positive region, it is widely used in deep networks. However, for negative inputs its gradient becomes zero, and some neurons may permanently output zero (dead neurons).

Leaky ReLU and why it is used in detection architectures

To reduce the dead-neuron issue, Leaky ReLU is often used, as also adopted in the YOLO paper [1]:

$$\phi(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases} \qquad (\alpha \approx 0.1).$$

In this case the negative region has a nonzero gradient, leading to more stable learning. This is particularly important in real-time applications, which often rely on lighter architectures and limited computational budgets.

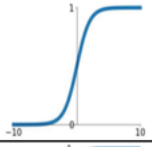
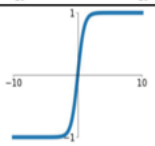
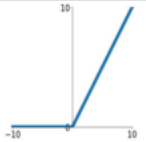
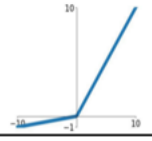
تابع فعالساز	رابطه	بازه خروجی	نمودار
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$[0,1]$	
Tanh	$\tanh(x)$	$[-1,1]$	
ReLU	$\max(0,x)$	$[0,+\infty]$	
Leaky ReLU	$\max(0.1x,x)$	$[-\infty,+\infty]$	

Figure 2.2: Comparison of common activation functions, including their formulas and output ranges.

2.3.3 Pooling and stride

Reducing the spatial dimensions of feature maps is useful for controlling computational cost, increasing the effective receptive field, and providing some invariance to small translations. Pooling (e.g., Max Pooling) summarizes local regions to reduce output size. Similarly, increasing Stride (the filter step size) reduces output resolution and can accelerate inference.

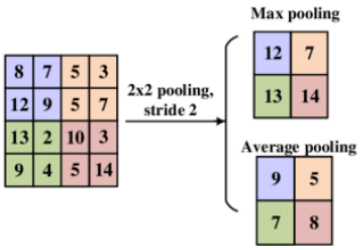


Figure 2.3: An example of downsampling via Pooling and the effect of Stride in reducing feature map size and increasing speed.

2.3.4 Flattening and fully connected layers

After extracting spatial features, Flattening is typically used to convert feature maps into a vector, enabling fully connected layers to perform final decisions (e.g., classification). However, in object detection we must not only classify objects but also localize them; therefore, we need architectures that perform localization and classification jointly.

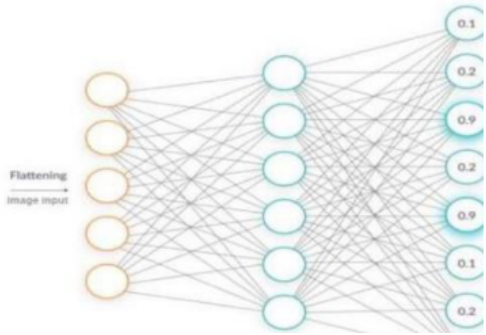


Figure 2.4: A high-level view of fully connected layers after Flattening for final decision-making.

2.4 Multi-stage object detection and an introduction to R-CNN

2.4.1 Core idea of R-CNN

The R-CNN (Regions with CNN features) framework is a landmark in object detection, demonstrating that combining region proposals with deep CNN features can achieve high accuracy [2]. The core idea is: first, class-agnostic Region Proposals are generated; then deep features are extracted for each proposal; finally, the object class and location are predicted.

2.4.2 Standard R-CNN pipeline

According to [2], the R-CNN detection pipeline proceeds as follows:

1) *Region proposal generation: methods such as Selective Search generate about* ~ 2000 *candidate regions.*

2) *Warp/size normalization: each region is resized to a fixed size (e.g.,* 227×227*) to match network input.*

3) *Feature extraction: each region is fed through the network to produce a feature vector (e.g., 4096 dimensions).*

4) *Classification: for each class, a linear SVM is trained:*

$$s_c(x) = w_c^\top \phi(x) + b_c.$$

5) *Suppress overlaps (NMS): repeated boxes are removed using the IoU criterion:*

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

6) *Bounding-box regression: to improve localization, a regression mapping is learned on the features. A common transform (as in the paper) is:*

$$\hat{G}_x = P_w\, d_x(P) + P_x, \qquad \hat{G}_y = P_h\, d_y(P) + P_y,$$

$$\hat{G}_w = P_w\, exp(d_w(P)), \qquad \hat{G}_h = P_h\, exp(d_h(P)),$$

where P is the proposal box, $\hat{G}$ is the refined box, and each $d_*(P)$ is typically modeled linearly from the features:

$$d_*(P) = w_*^\top \phi(P).$$

2.4.3 Why R-CNN is not ideal for real-time detection

Because R-CNN processes a large number of proposals independently, it incurs high computational cost. In online applications such as CCTV video monitoring, running a CNN on thousands of regions per frame introduces significant latency. Hence, despite its historical importance and reasonable accuracy, multi-stage approaches are often limited in real-time settings and motivate the move toward more integrated methods.

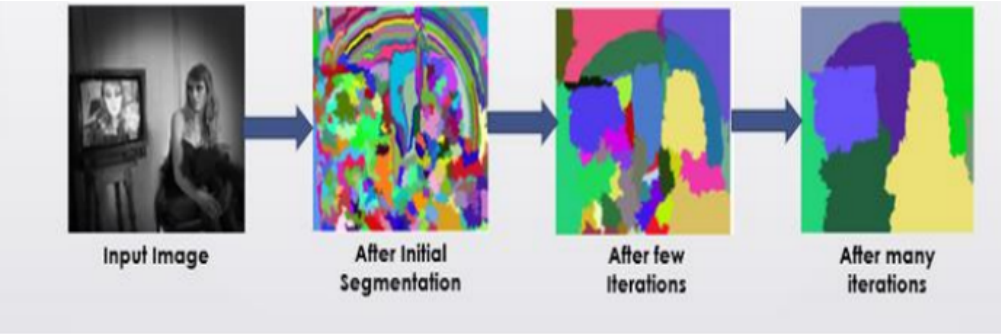


Figure 2.5: An example of multi-stage/iterative image processing: the input image and outputs after increasing iterations. Such iterative pipelines typically raise computational cost and are challenging for real-time deployment.

2.5 Transition to one-stage methods and choosing YOLO

2.5.1 The YOLO idea

In the YOLO family, object detection is treated as a unified problem: the image passes through the network only once, and the output simultaneously contains (1) bounding-box coordinates, (2) an objectness/confidence score, and (3) class probabilities [1]. This design reduces computational overhead and makes the model more suitable for online applications.

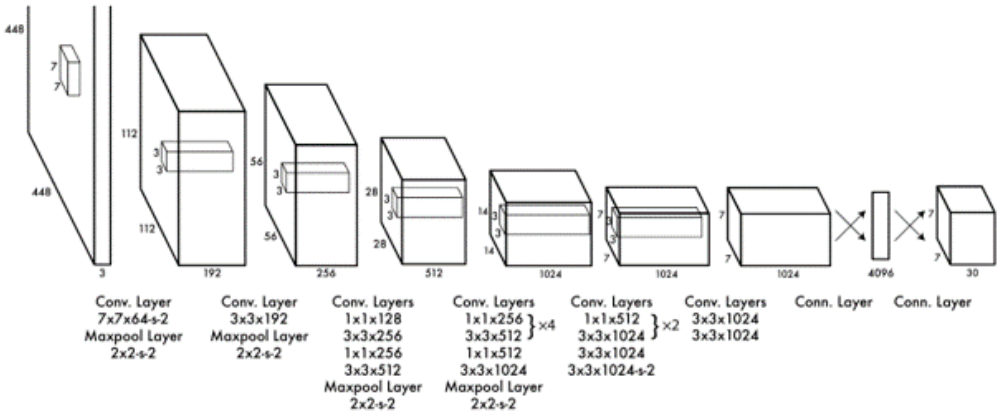


Figure 2.6: A schematic view of the YOLO architecture: the sequence of convolutional layers and how localization and classification outputs are produced.

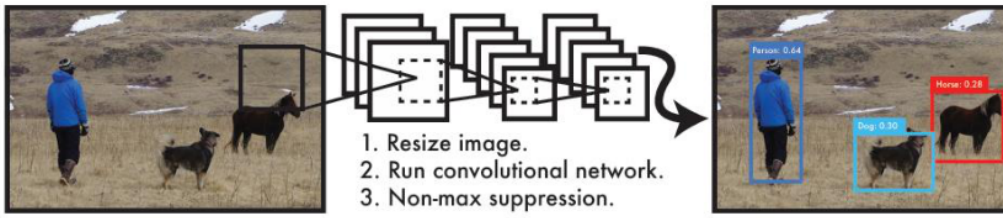


Figure 2.7: The overall YOLO pipeline: resizing the image, extracting features with a CNN, and removing overlaps using NMS.

2.5.2 YOLO loss function

According to the YOLO paper [1], the loss is typically multi-part so that localization error, objectness confidence, and classification error are controlled jointly. A standard form is:

$$\begin{aligned}\mathcal{L} &= \lambda_{coord} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ &+ \lambda_{coord} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbb{1}_{ij}^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ &+ \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_{noobj} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\ &+ \sum_{i=1}^{S^2} \mathbb{1}_{i}^{obj} \sum_{c \in classes} \left(p_i(c) - \hat{p}_i(c) \right)^2 \end{aligned} \tag{2.1}$$

where S^2 is the number of grid cells, B is the number of predicted boxes per cell, $\mathbb{1}_{ij}^{obj}$ indicates whether a box is responsible for predicting an object, and $\lambda_{coord}, \lambda_{noobj}$ are weighting coefficients to balance the terms.

2.6 Optimization and learning

After defining the loss, the network is trained using gradient-based methods: the loss gradient with respect to parameters is computed and parameters are updated in the opposite direction of the gradient. This process is repeated across epochs/iterations until $\mathcal{L}$ decreases.

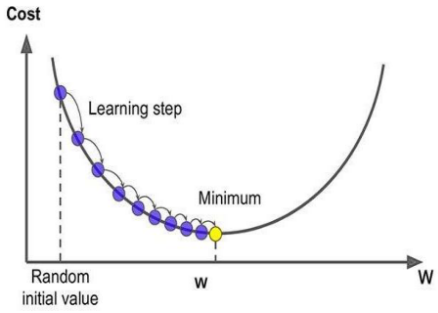


Figure 2.8: A schematic illustration of gradient descent moving toward a minimum of the loss function.

2.7 Chapter summary

This chapter first reviewed the components of convolutional networks and their role in extracting visual features. Nonlinear activation functions were then discussed in detail, highlighting why nonlinearity is critical for learning complex smoke and fire patterns. Next, the multi-stage R-CNN framework was introduced as an influential approach, and its limitations for online detection were explained. Finally, given the need for speed and integration, the rationale for selecting one-stage architectures such as YOLO was presented and the loss function was formulated. These concepts form the basis for the model selection and implementation described in subsequent chapters.

Chapter 3

Data Engineering and Dataset Development

3.1 Introduction and the need for a data-centric approach

The accuracy and stability of a smoke and fire detection system depend primarily on the quality and diversity of the training data. Smoke is a non-rigid and dynamic phenomenon whose appearance changes with environmental conditions (lighting, viewpoint, distance, background, and ventilation). Moreover, visual similarity between smoke and phenomena such as steam, fog, dust, and certain lighting patterns increases the likelihood of errors. Therefore, dataset design must cover realistic scenarios while also including difficult and underrepresented cases.

Accordingly, dataset development in this project is carried out as a staged, goal-driven process based on a combination of real data and generative (synthetic) data, so that the limitations of each source are compensated by the other.

3.2 Dataset design principles and coverage criteria

The objective of dataset development is to build a comprehensive dataset that stabilizes model behavior in the operational environment. For this purpose, the following criteria are considered as the main coverage axes:

- *Imaging condition diversity: day vs. night lighting, artificial light, shadows, reflections, and sudden illumination changes.*
- *Viewpoint and distance diversity: differences in camera mounting height, oblique*

viewing angles, near vs. far distances, and perspective variations.

- *Background and environment diversity: industrial settings, urban scenes, outdoor spaces, sheds, and semi-open areas with different textures and colors.*
- *Smoke shape and intensity diversity: thin and semi-transparent smoke in early stages, dense and thick smoke, and temporal variations in smoke flow.*
- *Challenging negative samples: visually smoke-like cases (steam, fog, dust, localized lighting, and contrast changes).*

These criteria align directly with the system's main goal: detecting smoke in its early stage is typically harder than detecting thick smoke and requires sufficient coverage of "small and faint smoke" appearances.

3.3 Limitations of real data and scarcity of key scenarios

In the first phase, real data is collected from available sources. While real data is essential for preserving realism and reducing the train–deploy gap, several structural limitations emerge:

- *A significant portion of available data corresponds to advanced stages of smoke and fire, whereas the system's goal is rapid detection in early stages with thinner smoke.*
- *Many existing images and videos are captured from standard cameras or handheld devices and do not perfectly match CCTV viewpoints.*
- *Some environments and backgrounds are repetitive; if not controlled, the model may overfit to background cues rather than learning smoke features.*
- *Specific events—very faint smoke, smoke under low light, smoke over bright backgrounds, smoke with strong motion, or smoke behind obstacles—are underrepresented in real data, and obtaining them via real-world collection alone is costly and time-consuming.*

Therefore, relying solely on real data is not sufficient for full coverage of the problem space, and a complementary solution is needed to increase diversity and cover rare scenarios.

Figure 3.1: An example real image of outdoor smoke (as a reference for background and lighting diversity).

Figure 3.2: Several sample frames extracted from available videos (e.g., public sources such as YouTube) to increase the diversity of real-world scenarios.

Figure 3.3: An example low-quality frame from a camera/video (illustrating noise and quality limitations in real data).

3.4 Real data collection and extraction of training samples

Despite limitations, real data remains the foundation of the dataset because it contains natural image characteristics, real camera noise, and genuine environmental patterns. In this stage, videos are used in addition to images so that temporal variations (smoke motion, shape changes across consecutive frames, and illumination changes) can be better represented. Suitable frames are then selected from videos and added to the dataset.

Figure 3.4: A collage of real samples used in the dataset (a high-level view of environment and imaging diversity).

3.5 Cleaning, standardization, and labeling

After initial collection, data enters a refinement stage to stabilize learning quality. Key actions include:

- *removing low-quality, blurry, duplicate, or heavily noisy images,*
- *standardizing image size/resolution and storage format for consistency,*
- *precise labeling: assigning classes and drawing bounding boxes for smoke/fire (when present),*
- *manual review of sensitive samples to reduce labeling error and prevent propagating human mistakes to the model,*
- *splitting data into training, validation, and test sets for reliable and repeatable evaluation.*

The output of this stage is a clean real dataset that acts as the "realism core" of the training process.

3.6 Hybrid approach: enriching the dataset with generative (synthetic) data

To fill the gaps of real data, synthetic data is generated using generative models. The goal is not to replace real data but to add a complementary layer that injects rare

and difficult scenarios into the dataset in a controlled manner. An important principle is to generate data purposefully, not merely to increase the number of images. Therefore, generation follows the same coverage criteria, including:

- *generating very thin and semi-transparent smoke to simulate early-stage fires,*
- *generating small-to-medium smoke regions to cover early events,*
- *diversifying backgrounds with emphasis on CCTV-like environments,*
- *creating challenging lighting conditions such as low light, strong light, shadows, and reflections.*

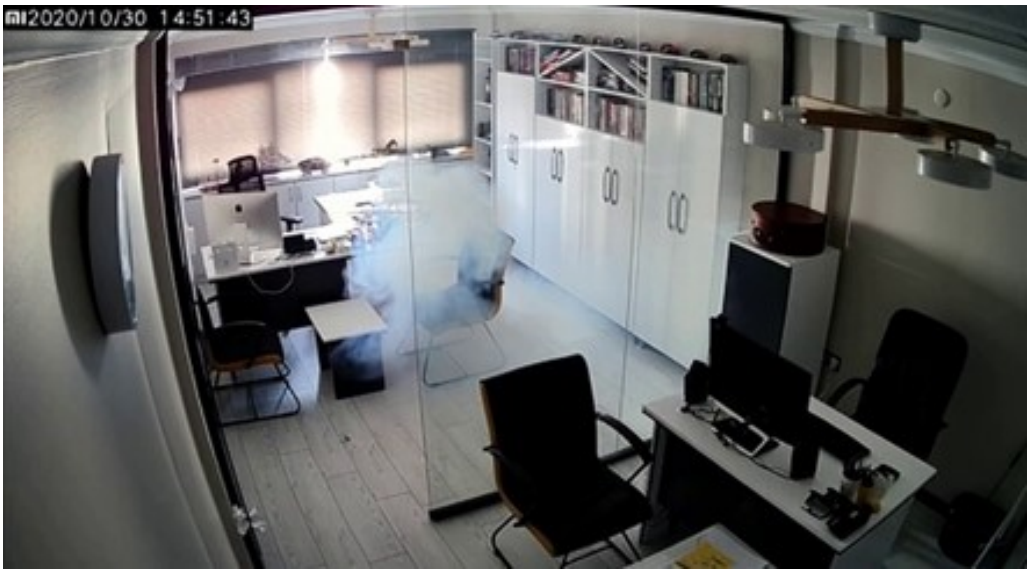


Figure 3.5: Examples of images generated by generative models (as a complement to real data for increasing scenario diversity).

Figure 3.6: Comparison between a real sample and a generated sample to illustrate the logic of targeted dataset enrichment.

Figure 3.7: A collage of generated samples after refinement (a high-level view of diversity in the synthetic data).

3.7 Quality control for generative data and preventing harmful samples

Generated data may contain visual artifacts, unrealistic structures, or repetitive patterns that can mislead the model if included in training. Therefore, quality control is essential. Key actions include:

- *removing samples with obvious artifacts or unrealistic smoke,*
- *removing samples that depict smoke in physically implausible or inconsistent ways,*
- *removing samples whose artificial backgrounds or incorrect compositing may introduce bias,*
- *ensuring sufficient diversity and avoiding repeated patterns in the generative outputs.*

After refinement, generative data is incorporated into the final dataset as a complement to real data.

3.8 Final dataset composition and rationale for mixing ratios

In the final stage, the dataset is built as a hybrid. Real data stabilizes realism and transfers natural environmental features, while generative data covers underrepresented

scenarios and provides targeted diversity. The mixing ratio should be chosen so that:

- *the model is not dominated by unrealistic data or fabricated patterns,*
- *while rare and difficult scenarios are strengthened sufficiently.*

From this perspective, generative data is most useful when it directly addresses identified gaps—i.e., the exact weaknesses where real data is scarce.

3.9 Benefits of the data-centric approach and its impact on model performance

The combined use of real and generative data provides several practical advantages:

1. *Improved generalization: the model observes a wider range of environments instead of depending on a few backgrounds.*
2. *Better early-stage coverage: thin and small smoke cases that are rare in real data are strengthened in a targeted way.*
3. *Reduced data bias: diversifying lighting and environments reduces the risk of learning spurious correlations.*
4. *Lower collection cost and time: many specialized scenarios can be produced without costly field collection.*
5. *Increased readiness for deployment: exposure to higher diversity improves stability under real CCTV conditions.*

3.10 Chapter summary

This chapter showed that relying on real data alone is insufficient to cover all operational scenarios, and that a hybrid strategy (real + generative) can compensate for missing key cases. In this framework, real data provides the "core realism," while generative data provides "targeted diversity expansion." Moreover, through careful cleaning, standardization, labeling, and quality control of generative samples, the final dataset can be constructed to support stable training and to improve model generalization in real industrial environments.

Chapter 4

Model Training and Empirical Evaluation

4.1 Design of training scenarios

To comprehensively evaluate the performance of the proposed system and analyze the effects of model architecture and data/training conditions on smoke and fire detection accuracy, four distinct training scenarios were designed and executed. The experimental design aimed to answer several key questions: (1) Which configuration in the YOLO family provides a better trade-off between speed and accuracy for smoke and fire detection? (2) Does increasing model capacity (using larger variants) necessarily improve evaluation metrics, or can performance degrade due to data limitations and complexity? (3) How sensitive is the model to different training/data conditions, and can results change significantly even when the architecture is fixed?

Across all scenarios, training was conducted for 70 epochs with batch size = 16. Final evaluation was performed on a test set containing 42 images and 55 labels. The metrics mAP@50, mAP@50:95, Precision, and Recall were selected as primary indicators to jointly analyze localization quality, detection quality, and the balance between false alarms and missed detections.

4.2 Description of the executed scenarios

4.2.1 Scenario 1: YOLOv11-n

In this scenario, the lightweight YOLOv11-n variant was used. The main motivation was to obtain a deployable option for real-time processing in industrial environments, where hardware constraints and the need for timely alarms are critical. After training

with the stated configuration, the model was evaluated on the test set and quantitative results were recorded.

4.2.2 Scenario 2: YOLOv11-m

This scenario uses the larger YOLOv11-m variant to examine the effect of increased capacity and parameter count. A natural expectation is that larger models can learn more complex patterns if sufficient diverse data is available; however, for smoke (especially thin smoke with ambiguous boundaries and similarity to confounders), higher capacity can also increase sensitivity to data conditions. Therefore, this scenario assesses the practical benefit of the larger variant and its deployment feasibility.

4.2.3 Scenario 3: YOLOv9-e

Here, YOLOv9-e was trained to enable comparisons across different generations of the YOLO family. This helps determine whether architectural changes across generations yield meaningful improvements for smoke/fire detection. In addition, earlier generations can be more established and stable in practice, so evaluating them is valuable for final selection.

4.2.4 Scenario 4: YOLOv11-n (different conditions)

This scenario again uses YOLOv11-n, but under different training/data conditions compared to Scenario 1 (to assess sensitivity). The goal is to show that even with the same architecture, differences in training conditions—such as data type and quality, domain alignment, noise/frame quality, or labeling and cleaning differences—can lead to noticeable changes in final metrics. This analysis is important for practical decisions about data engineering and deployment stability.

4.3 Quantitative results and comparison

To compare the scenarios, four main metrics are reported. mAP@50 reflects detection quality at IoU=0.5, while mAP@50:95 averages performance across multiple IoU thresholds and is stricter. Precision and Recall represent the correctness of positive alarms and the ability to discover true instances, respectively. Table 4.1 summarizes the evaluation results.

Table 4.1: Summary of evaluation results for different models

Scenario	*Model*	*mAP50*	*mAP50-95*	*Precision*	*Recall*
1	*YOLOv11-n*	*83.0*	*39.0*	*81.0*	*71.0*
2	*YOLOv11-m*	*69.0*	*33.0*	*61.0*	*69.0*
3	*YOLOv9-e*	*78.0*	*33.0*	*82.0*	*68.0*
4	*YOLOv11-n*	*41.0*	*19.0*	*59.0*	*30.0*

4.4 Analysis and discussion

Based on Table 4.1, Scenario 1 with YOLOv11-n provides the best overall performance in terms of mAP50 (83.0), and the Precision and Recall values indicate a reasonable balance between false-alarm rate and true-instance discovery. This suggests that the lightweight variant, combined with appropriate training settings, can achieve strong accuracy and is a reliable candidate for real-time deployment.

In Scenario 2, the larger YOLOv11-m model does not yield a noticeable improvement despite higher capacity. This may be due to factors such as the greater need of larger models for more diverse data, higher sensitivity to overfitting, or the difficulty of learning smoke boundaries (especially thin smoke). Additionally, higher capacity typically increases computational cost; therefore, if it does not provide a meaningful accuracy benefit, selecting a lighter model is more sensible.

Scenario 3 with YOLOv9-e shows competitive performance and achieves a high Precision, but it does not match Scenario 1 in terms of mAP50. This can indicate that, for this task, architectural improvements in the newer generation (v11) in the lightweight form provide a better trade-off.

Importantly, Scenarios 1 and 4 both use YOLOv11-n, but due to differences in training/data conditions, their results differ substantially. This clearly shows that final performance is not only a function of "model name." Factors such as data quality, domain alignment with the target environment, noise and frame quality, and labeling accuracy can have decisive effects. Therefore, alongside selecting an appropriate architecture, data engineering and dataset quality control are critical for achieving stable, deployable performance.

Overall, considering the reported metrics and operational constraints (speed, computational cost, and deployability), YOLOv11-n in Scenario 1 is recommended as the

more suitable option for deploying an online smoke and fire detection system.

4.5 Sample outputs

Figure 4.1: Several frames from a video

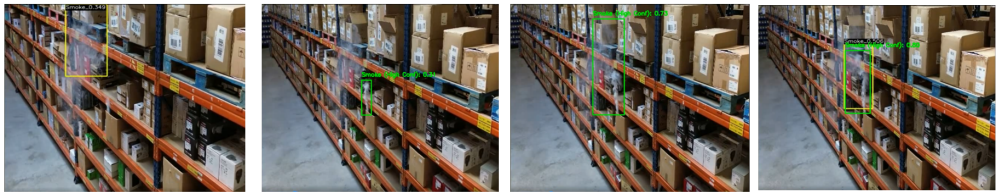


Figure 4.2: Several frames from a video

Figure 4.3

Figure 4.4

Chapter 5

Video-based Detection and Optimization for Continuous Monitoring

5.1 Introduction

In industrial applications, the output of a smoke detection system should not be based only on a single image; rather, it must operate as continuous, real-time monitoring over the CCTV video stream. In such conditions, one of the main challenges of vision-based systems is the occurrence of False Positive alarms due to visual similarity between smoke and certain environmental phenomena or objects. Practical observations show that a model may become biased toward patterns such as bright clouds in the sky, specular reflections, or even light and compact objects such as pieces of plastic bags, and mistakenly report them as smoke. If not controlled, this increases the false-alarm rate and reduces system reliability in the operational environment.

Figure 5.1: An example of a crumpled plastic object that the model may incorrectly detect as smoke (a false alarm caused by visual similarity).

Given the nature of the problem and the available infrastructure, two key properties can be exploited to control errors:

- *The input is CCTV video, which provides temporal information.*
- *Smoke is inherently dynamic and non-rigid, and its spatiotemporal behavior differs from many static objects or smoke-like patterns.*

Based on this, this chapter proposes a multi-frame temporal analysis (Temporal Consistency) approach whose goal is to reduce false alarms without significantly increasing computational cost.

5.2 Core idea: separating real smoke from visual biases using spatial dynamics

Smoke typically changes shape over time, expands, moves, or at least exhibits fluctuations along its boundaries. In contrast, many causes of false alarms—such as a relatively static cloud in a small part of the sky, fixed bright spots, or an object like a plastic bag that stays in nearly the same position over several consecutive frames—show different spatial stability. Therefore, even if the detector produces high confidence for a region in a single frame, analyzing the changes in the detection box location and shape across multiple frames can be a useful criterion to distinguish "real smoke" from "bias-inducing patterns."

To formalize this idea, consider the detected box in frame t as

$$b_t = (x_t, y_t, w_t, h_t),$$

where (x_t, y_t) is the box center and w_t, h_t are width and height. Over a temporal window of N frames, spatial/shape variations can be quantified, for example, by the average center displacement and size changes:

$$\Delta_c = \frac{1}{N-1}\sum_{t=2}^{N}\sqrt{(x_t - x_{t-1})^2 + (y_t - y_{t-1})^2}, \quad \Delta_s = \frac{1}{N-1}\sum_{t=2}^{N}\left(|w_t - w_{t-1}| + |h_t - h_{t-1}|\right).$$

Intuitively, real smoke usually produces larger Δ_c and/or Δ_s than quasi-static smoke-like objects (e.g., fixed bright spots or the crumpled plastic in Fig. 5.1).

5.3 A two-threshold confidence strategy and multi-stage decision making

To implement the above idea, a decision mechanism with two empirical thresholds is defined:

- *High threshold T_h: if detection Confidence exceeds this value, the detection is accepted as smoke. The rationale is that some cases are sufficiently clear and do not require temporal verification; rejecting them may increase False Negative errors.*
- *Low threshold T_l: if confidence is below this value, the output is not reported as smoke. This removes weak predictions and reduces output noise.*
- *Middle range $[T_l, T_h]$: in this range, the final decision depends on temporal analysis; the output is not accepted based on a single frame and must be verified across consecutive frames.*

This approach lets the model act quickly on "clear" cases, suppress "weak" cases, and use multi-frame analysis only for "borderline/suspicious" cases to increase confidence.

5.4 Temporal criterion: analyzing box position changes across multiple frames

Within the middle range, a short temporal window of N consecutive frames is considered. For each frame, the detection box and confidence are extracted. A simple yet effective dynamics criterion is then applied:

- *If the box remains nearly stationary over the window (i.e., its center and dimensions change only slightly), the likelihood of true smoke decreases and the output is rejected as bias or smoke-like object.*
- *Conversely, if meaningful changes in position/size/boundary are observed across several frames, the output is more likely to be smoke and is accepted.*

In a compact formulation, acceptance can be defined via dynamic thresholds:

$$\textit{Accept if} \quad \Delta_c \geq \delta_c \;\; \textit{or} \;\; \Delta_s \geq \delta_s,$$

where δ_c, δ_s are empirical thresholds tuned based on frame rate and image scale. This temporal criterion effectively acts as an anti-false-alarm filter, particularly for cases such as clouds, fixed bright spots, or relatively stationary objects whose appearance can be mistaken for smoke.

5.5 Reducing computational cost via frame sampling and batch inference

From an operational perspective, processing all video frames at full rate can increase hardware cost. To control this, two complementary strategies can be used:

1. *Temporal sampling (Frame Skipping): instead of processing every frame, process one frame out of every k. Since the proposed criterion relies on overall changes across multiple frames, in many scenarios a reduced processing rate still provides sufficient temporal information.*
2. *Batch inference (Batch Inference): if supported by the hardware/software stack, multiple frames can be fed to the model simultaneously to improve GPU utilization and reduce total time. This is especially useful when monitoring many cameras or multiple video streams in parallel.*

Combining these strategies with the two-threshold mechanism reduces computational cost because multi-frame analysis is activated only for a small subset of predictions (borderline cases) rather than for all outputs.

5.6 Summary

To improve system reliability in video monitoring, this chapter presented a temporal-analysis approach aimed at reducing false alarms caused by visual biases such as clouds, reflections, and smoke-like objects. Using two confidence thresholds and multi-stage decision making, clear cases are accepted quickly, weak cases are suppressed, and only borderline cases trigger multi-frame verification. The key criterion is the spatial dynamics of detection boxes across consecutive frames: smoke is non-rigid and variable, while many false-alarm sources are spatially more stable. Finally, computational cost can be reduced via frame sampling and batch inference, keeping the system suitable for real-time deployment while lowering the false-alarm rate.

Chapter 6

Conclusion

This study investigated early smoke and fire detection in industrial environments using a computer vision and deep learning approach. The system design was driven by two primary needs: first, increasing detection speed in the early stage of smoke formation; and second, enabling practical deployment at reasonable cost by leveraging existing infrastructure such as CCTV cameras. Accordingly, the work began by reviewing convolutional networks and their ability to extract visual features, and then—given real-time constraints—moved away from region-based, multi-stage architectures toward selecting a fast, unified object detection architecture. Within this framework, YOLO was introduced as a suitable option for online analysis, and its training process was explained through loss minimization and gradient-based optimization/backpropagation.

Next, the research focused on the key factor that largely determines the performance of vision models: data. Due to limitations in real data (especially scarcity of thin-smoke samples, limited viewpoint and lighting diversity, and insufficient CCTV-aligned samples), a hybrid dataset was designed as a combination of real data and controlled synthetic data produced by generative models. The aim was to cover rare and difficult scenarios and reduce data bias without sacrificing realism. Along with quality control, this data engineering enabled more stable training and better model generalization.

Empirical evaluations showed that among the explored training scenarios, YOLOv11-Nano provides the best balance between detection accuracy and deployability. Compared to larger variants, the Nano model not only achieved stronger quantitative performance but, due to its lighter capacity, was less prone to overfitting and more suitable for online applications. In addition, comparison against a larger public dataset highlighted that dataset quality and domain alignment can be more decisive than simply increasing data volume, and can lead to better performance on evaluation metrics.

Beyond image-based evaluation, the system was also extended to video monitoring. Given smoke's dynamic nature, a temporal-analysis mechanism based on spatial stability of detection boxes was proposed to reduce false alarms caused by visual biases such as clouds, reflections, or smoke-like objects. This temporal mechanism significantly reduced false alarms in video streams and improved output stability compared to single-frame decisions. Overall, this work demonstrates that combining (1) an architecture suitable for real-time inference, (2) targeted data engineering via a hybrid dataset, and (3) video-level decision optimization can deliver a high-quality, practically deployable smoke and fire detection system.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," arXiv preprint arXiv:1506.02640, 2016.

[2] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," arXiv preprint arXiv:1311.2524, 2014.